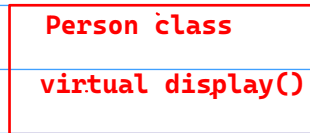
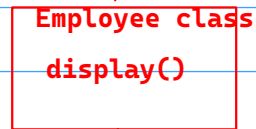


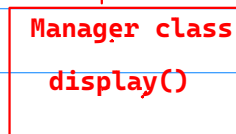
Multilevel Inheritance



Note:-If a function is declared as virtual in the base class, it will be virtual in all its derived classes.



virtual function



virtual function

menu driven program with pure virtual function

Abstract base class → Shape

virtual void calArea()=0;

pure virtual function

Circle

```
void calArea()
{
}
```

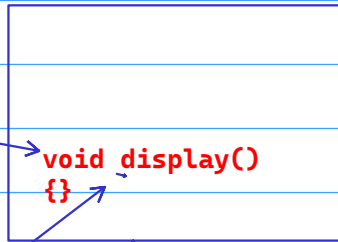
Rectangle

```
void calArea()
{
}
```

```
shape * sh;
Circle c1;
Rectangle r1;
```

```
Enter which shape
are ayou want to
calculate
1. Cal Area of Circle
2. Cal Area of Rectangle
switch(ch)//1
{
    case 1:
        sh=&c1;
    brea;
    case 2:
        sh=&r1;
    break;
}
sh→calArea();
```

preson



non virtual base class
member function

Early binding

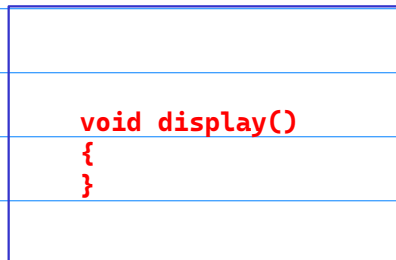
Employee e1;

//upcastoing

Person * ptr=&e1;

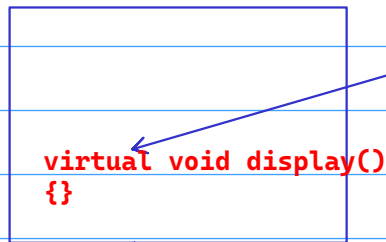
ptr→ display();
//it will call base class
//function

Employee class



Note:- function call is getting resolved during compile time and it look towards the data type of pointer ptr and it will the base class version of display()

person



virtual base class
member function

Late binding

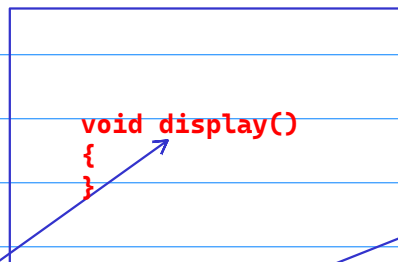
```
Employee e1; // derived class obj
Student s1; // derived class obj
```

```
// upcasting
Person * ptr;
```

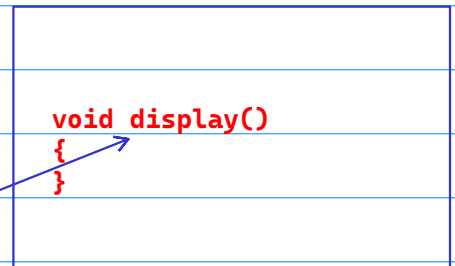
```
1. Employee obj
2. Student obj
enter the choice in ch
// 1
switch(ch)
{
```

```
    case 1:
        ptr=&e1;
        ptr->display();
        break;
    case 2:
        ptr=&s1;
        ptr->display();
        break;
}
```

Employee class



Student



Note:- here function call is getting resolved at runtime and it is looking towards the address of which derived class object is stored in base class pointer and as per the type of address of derived class obj it is calling the respective class function.

```
ptr-> display();
//it will call base class
//function
```